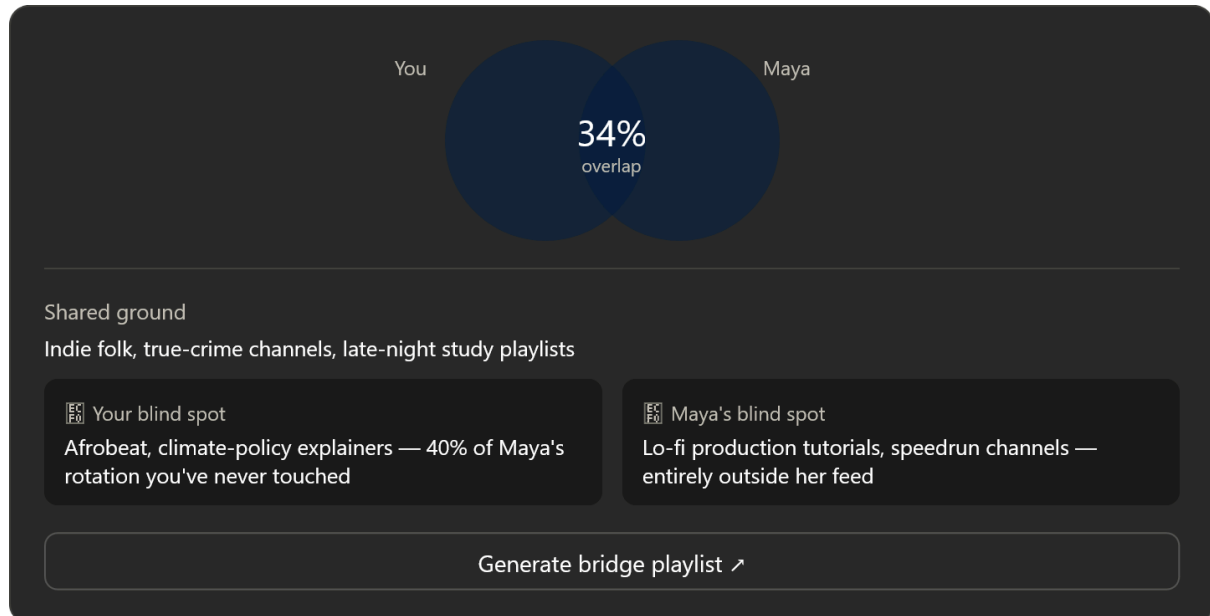


Project proposal: Tamago

One-line pitch

A stranger-matching chat app — like Omegle — where pairing isn't random. You're matched by taste, using a compatibility score built from your actual Spotify and YouTube history, revealed the moment you connect.



Problem statement

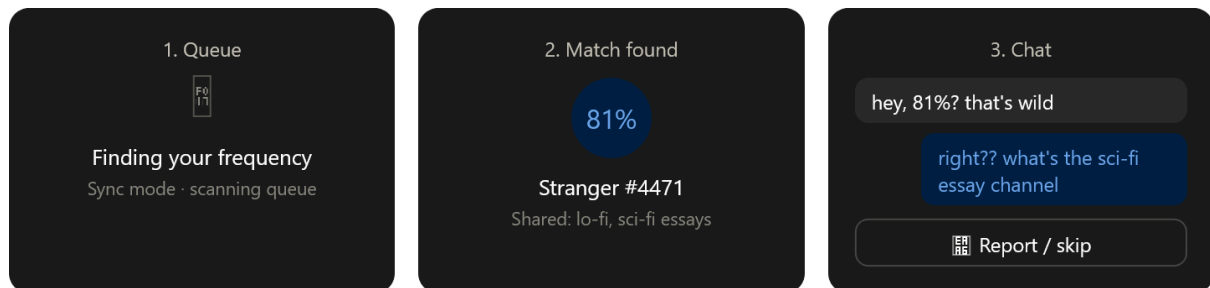
Random stranger-chat apps pair people with zero context, which is why most conversations die in the first ten seconds. Meanwhile, platforms already know exactly what would make two strangers click — they just never use that insight to introduce people, only to sell ads. This project repurposes that same signal for something more interesting than advertising: giving two strangers a reason to talk.

Core concept

3 matchmaking modes:

- **Sync mode** — paired with your closest taste match currently in the queue. Find your "frequency twin."
- **Shuffle mode** — deliberately paired with someone whose taste is far from yours. Guaranteed exposure to something outside your bubble, framed as a discovery mode rather than a compatibility one.
- **(Update)** Search/filter by keywords/types that users want based on music taste (ignore compatibility %), e.g., softie/cutie, badie,...

The moment a match connects, both people see a compatibility card before the conversation starts: overlap %, shared interests, and each person's blind spots relative to the other. This reuses the same feature-extraction engine from the Digital Shadow / Shadow Match concept — one pipeline, now powering a live social feature instead of a static dashboard.



How it works

1. **Taste vector** — each user's Spotify/YouTube history is reduced to an anonymized embedding (genre distribution, topic clusters, mood profile). Only this vector is ever transmitted — never raw listening history.
2. **Matchmaking queue** — users entering the queue submit their vector. A lightweight matching pass computes cosine similarity against everyone currently waiting and pairs by threshold: closest for Sync, farthest for Shuffle.
3. **Connection** — once paired, a real-time chat session opens (Socket.io or WebRTC — well-established patterns, no need to build signaling from scratch).
4. **The reveal** — compatibility card renders at connection: overlap score, shared clusters, blind spots. This is the hook — it gives strangers an immediate icebreaker instead of "a/s/l."

Data sources

- **Spotify**: Extended Streaming History export → track/artist/timestamp data. Audio features (valence, energy, danceability) via the Web API.
- **YouTube**: Google Takeout [watch-history.json](#) → titles/timestamps, clustered into topics via keyword/TF-IDF matching (or the YouTube Data API for categories if time allows).
- Only the derived, anonymized vector leaves the client — raw data stays local.

Tech stack

- **Data processing**: Python (pandas for parsing, simple clustering for topic/genre vectors)
- **Matchmaking server**: lightweight queue + periodic similarity-pairing pass
- **Real-time chat**: Socket.io or WebRTC for the paired session
- **Frontend**: queue screen → match/compatibility reveal → chat interface
- **Coding execution**: Gemini

Build priority (time-boxed)

1. Taste vector extraction from Spotify data (fastest, most reliable signal)
2. Static compatibility card between two known users — prove the core mechanic works before adding real-time matching
3. Matchmaking queue + similarity pairing (start with Sync mode only)
4. Real-time chat connection
5. Shuffle mode + YouTube integration (stretch goals if time allows)

Demo flow for judges

Two teammates join the queue live from their own phones, get paired, and watch the compatibility card render in real time before they start chatting — the reveal moment doubles as the pitch. If live pairing is risky right before judging, have a pre-recorded backup of the same flow.

Safety and ethics (worth its own slide)

Stranger-matching apps live or die on moderation, and naming that risk directly is stronger than pretending it doesn't exist:

- Visible **report/skip button** in every session
- No raw personal data ever transmitted — only the derived anonymized vector
- MVP demo scoped to your own team/classmates, not the open internet
- Explicit note that production deployment would require moderation tooling and age-gating before any public release